

Backend Engineering

Workshop • Session 2 of 2

Recap

Session 1 gave us the shape of the job: APIs, data, authentication, background work, integrations, reliability, and infrastructure.

Today

Technology choices • AI • The backend market • A practical learning path

The Five Responsibilities Don't Change

The tool you use to do them does

The responsibility

Build APIs, work with data, secure access, run background jobs, integrate s

The ecosystem

Language + framework + database + cache/queue + infrastructure.

Don't ask only: "Which language is best?"

Ask: "Which ecosystem fits this kind of work and this kind of company?"

Backend Technologies

Each language has strengths and trade-offs

JavaScript / TypeScript

Startups, real-time apps, same language front-to-back
Trade-off: Huge ecosystem; package quality varies

C# (.NET)

Enterprise, Windows-integrated systems
Trade-off: Strong tooling; jobs skew corporate

Java

Large, long-lived enterprise systems
Trade-off: Verbose; stability is a major strength

PHP

Web-first, CMS, e-commerce
Trade-off: Modern PHP/Laravel is solid; old reputation remains

Python

Data-heavy APIs, ML/AI-adjacent backends
Trade-off: Readable; raw performance can be a weakness at scale

Go

Cloud-native, high-concurrency infrastructure
Trade-off: Smaller ecosystem; more building from scratch

Language Choice & Career Context

Look at the kind of companies and problems around the ecosystem

Node.js / TypeScript

Startups • product companies • freelance • real

PHP

Agencies • web shops • Laravel product teams •

C# / Java

Banks • enterprise software • telecom • large organizations

Python / Go

AI/data-heavy startups • scientific/fintech • cloud/infra companies

There is no single “winning” language — there is a fit between language, ecosystem, company, and problem.

Task 1

1

10 min • Small groups • Let's practice

Your group gets one language.

Search: “[language] backend developer jobs [your country/remote]” and “[language]

Report ONE surprising finding.

Task 2

2

5 min • Individual → pairs • Let's practice

**Write your current language/framework and ONE Session 1 responsibility you feel we
Compare with a partner.**

Backend Engineering in the Era of AI

What changes — and what doesn't

AI changes

Boilerplate and first drafts • faster lookup • draft code review and obvious

AI does not remove

Ownership • judgment about data • debugging genuinely unfamiliar production p

Implementation can become faster, while engineering judgment becomes more valuable.

Why Ownership Still Matters

Someone must make the decisions and own the outcome

Data decisions

What should we store? How does it relate? How

Production incidents

What happens when a novel failure appears in a

Operational responsibility

Someone still has to understand the system when it breaks

AI can propose an implementation. Engineering still requires understanding its consequences.

Task 3

3

8 min • Whole room • Let's practice

Name ONE Session 1 responsibility AI tools could compress the most.
Name ONE responsibility AI tools barely touch.
We'll collect answers in two columns.

Task 4

4

5 min • Pairs • Let's practice

Give an AI coding assistant an underspecified prompt such as:

“Add authentication to this endpoint.”

Then ask: “What decision did it make for you that YOU would want to make yourself?”

The Backend Development Market

Learn to read the market instead of memorizing a “hot language”

Job postings

Search real listings in your target region.

Remote vs. on-site

Check how roles are structured where you want to work.

Seniority

Compare junior, mid, and senior requirements separately.

Job title ≠ scope

A backend listing may include DevOps or full-stack work.

Task 5

10 min • Individual research • Let's practice

**Find THREE real backend job postings you would consider.
For each: note languages/frameworks, Session 1 responsibilities, and remote/hybrid/
Then compare: what appeared in all three?**

A Backend Learning Path

Think in sequences, not in a readers list

1

Fundamentals first

Programming • OOP • Data Structures • Algorithms

2

One language + one framework

Go deep before framework-hopping

3

Databases deliberately

Schema design + queries + evolving data

4

Build a real project

Auth + background work + integration + deployment

5

Systems thinking

System design + scaling + reliability

The Common Trap

Skipping foundations to reach “impressive” technology faster

Framework hopping

Five stacks shallowly can feel productive while le

Skipping databases

Query syntax is not the same as schema design c

Reading ≠ building

A queue can make sense on paper; building one creates

The goal is depth + real experience, then systems thinking.

Task 6

6

7 min • Pairs • Let's practice

Fictional learner:

“Knows basic JavaScript, built one small CRUD app, no database experience yet.”

Sequence their NEXT 3 steps using the 5-step learning path. Explain why.

Q / A

Let's turn today's ideas into practical decisions

Is it too late to start backend engineering because of AI tools?

Should I specialize in one language early, or stay flexible?

How can I tell if a job posting really wants a junior?

Your questions

What do you want to understand before you leave?

Workshop Closing

Two sessions. One bigger picture.

Session 1

The shape of the job: responsibilities, ownership, foundations, and stack.

Session 2

The terrain: technology choices, AI, the market, and a practical learning path.

Session 1 gave you the shape of the job.

Session 2 gave you the terrain it's played on.

Session Timing

Designed for approximately 75–90 minutes

Section	Content	Task	Subtotal
Recap bridge	2 min	—	2min
1. Technologies & careers	10 min	15 min	25 min
2. Backend in the era of AI	8 min	13 min	21 min
3. Development market	6 min	10 min	16 min
4. Learning path & resources	6 min	7 min	13 min
5. Q/A	10–15 min	—	10–15 min